Université Libre de Bruxelles

Lattice-based cryptanalysis of discrete-log signature schemes under partial nonce leakage

Student:
Ilias Elal-Lati
ULB ID: 000607880
Mohamed Amine Belmahi
ULB ID:000509057
David Botton
ULB ID: 000615056
Léandre Gerday
ULB ID: 000616822

Teachers:
Christophe Petit
Liran Lerman

April 28, 2026

Contents

1 Introduction and problem statement **2**

2 Experimental reproduction **2**

3 Technical background and paper contributions **2**

4 Experimental results and analysis **4**

5 Implementation challenges **5**

6 Discussion **5**

7 References **7**

1 Introduction and problem statement

Digital signature systems based on the discrete logarithm problem, such as the Digital Signature Algorithm (DSA) and the Elliptic Curve Digital Signature Algorithm (ECDSA), derive their security from the computational difficulty of solving discrete logarithms (Nguyen & Shparlinski, 2002; Boneh & Venkatesan, 1996). In these protocols, each signature requires an ephemeral secret value k that needs to be selected uniformly at random from $\mathbb{Z}_q^*$, with q representing the order of the relevant subgroup. Keeping both the secrecy and uncertainty of these nonce values is important, as any exposure or bias in their selection can enable adversaries to reconstruct the long-term private key α.

Nguyen and Shparlinski established that when an attacker possesses even partial information about the nonces across a sufficient set of signatures, it becomes possible to recover the private key in polynomial time.

The effects of these attacks in real life environments has been demonstrated by later research. Breitner and Heninger, for instance, used lattice-based analysis on publicly available ECDSA signatures obtained from major blockchains (such as Bitcoin, Ethereum and Ripple) and from Internet-wide scans (including SSH and HTTPS). They found that biases in nonce generation can allow attackers to recover private keys.

This project investigates how lattice-based cryptanalysis can break digital signature schemes through such weaknesses. It includes hands-on experiments that replicate simplified versions of real attacks using tools like SageMath and assesses both the strengths and limitations of these methods.

2 Experimental reproduction

So far, we have implemented a reproduction of the lattice-based attack on ECDSA with partially known nonces.

We have developed a Python/SageMath codebase (available at `https://github.com/nottoBD/lattice-biased-nonce-ecdsa`) that generates synthetic ECDSA signatures on the secp256k1 curve with controlled least-significant-bit (LSB) nonce leakage. For each generated dataset, we verify that the known portion of the nonces matches the expected biased values.

We have implemented the computation of the hidden-number problem quantities t_i and u_i as defined in Nguyen and Shparlinski's paper. Finally, we have built and executed a Boneh-Venkatesan-style lattice attack using LLL reduction followed by Babai's nearest-plane algorithm.

Our implementation successfully recovers the correct private key in validated test cases, notably when 8 least significant bits of each nonce are known and a sufficient number of signatures is available.

3 Technical background and paper contributions

We have studied the two foundational papers by Nguyen and Shparlinski on lattice attacks against DSA and ECDSA with partially known nonces, the seminal work by Boneh and Venkatesan that introduced the Hidden Number Problem (HNP) and the more recent practical application by Breitner and Heninger.

The core technical contribution of Nguyen and Shparlinski begins from the standard ECDSA signature equations. Let $\alpha \in \mathbb{F}_q^*$ be the signer's secret key and G a base point of prime order q.

For a message m, the signer chooses a random nonce $k \in \mathbb{F}_q^*$ and computes the signature (r, s) where

$$r = x(kG) \bmod q, \qquad s = k^{-1}\big(h(m) + \alpha r\big) \bmod q,$$

with $x(\cdot)$ denoting the x-coordinate of a point on the elliptic curve and h a cryptographic hash function. Rearrangement of the second equation immediately gives the linear relation

$$ks \equiv h(m) + \alpha r \pmod q.$$

Suppose that, for d signatures, the attacker knows the ℓ least significant bits of each nonce k_i. Then $k_i = k_i' + 2^\ell \tilde{k}_i$ where k_i' is known and $0 \le \tilde{k}_i < 2^{-\ell} q$. Substituting this expression and rearranging produces a collection of approximate modular linear relations

$$\alpha\, t_i \approx u_i \pmod q,$$

where the quantities t_i and u_i are efficiently computable from the public values r_i, s_i, k_i' and $h(m_i)$, and the approximation error is bounded by $q/2^\ell$. This is precisely a variant of the Hidden Number Problem (HNP) introduced by Boneh and Venkatesan.

Nguyen and Shparlinski prove that such an ECDSA-HNP instance can be solved in polynomial time using lattice reduction. They construct a lattice of dimension $d + 1$ in which sufficiently short vectors correspond directly to the unknown private key α. The LLL algorithm followed by Babai's nearest-plane method (a polynomial-time estimation algorithm for the Closest Vector Problem) is then applied to recover a short enough vector from which α can be extracted. The authors prove polynomial-time key recovery when roughly $\sqrt{\log q}$ nonce bits are known, up to logarithmic factors, and a sufficient number of signatures is available, under assumptions on the hash function and the distribution of the ECDSA values.

In our implementation, we have reproduced this exact lattice construction on the secp256k1 curve ($q \approx 2^{256}$). The code computes the HNP quantities t_i and u_i as:

$$t_i = 2^{-\ell} \cdot r_i \cdot s_i^{-1} \bmod q, \qquad u_i = 2^{-\ell} \cdot \big(k_i' - s_i^{-1} h(m_i)\big) \bmod q,$$

where k_i' is the known LSB part of nonce k_i and $2^{-\ell}$ denotes the modular inverse of 2^ℓ.

Following the Boneh-Venkatesan reduction, we then form the approximation $v_i = u_i + q/2^{\ell+1}$ and build the $(d+1)$-dimensional integer-scaled basis, where d is the number of signatures used:

$$B = \begin{pmatrix} q^2 & 0 & \cdots & 0 & 0 \\ 0 & q^2 & \cdots & 0 & 0 \\ \vdots & & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & q^2 & 0 \\ qt_1 & qt_2 & \cdots & qt_d & 1 \end{pmatrix}$$

with target vector $\mathbf{c} = (qv_1, \ldots, qv_d, 0)$. Scaling by q keeps all entries integral so that SageMath's LLL routine can be applied directly. After LLL reduction, Babai's nearest-plane algorithm is applied to approximate the closest lattice vector to $\mathbf{c}$. The last coordinate of this vector yields a candidate for $\alpha \bmod q$.

The mathematical core of the attack relies on the fact that the target vector $\mathbf{c}$ is unusually close to a specific hidden vector within the lattice. By definition of the lattice B, any vector generated by it takes the form $v_{\text{lattice}} = \alpha \cdot \text{row}_{d+1} - \sum z_i \cdot \text{row}_i$. The specific lattice vector

corresponding to the private key α has coordinates roughly equal to $q\alpha t_i \pmod{q^2}$.

Because the approximation error for each signature is bounded by $|\alpha t_i - v_i|_q \leq q/2^{\ell+1}$, the infinity norm of the distance between the target vector $\mathbf{c}$ and this hidden lattice vector is strictly bounded by $q^2/2^{\ell+1}$. When ℓ is large enough, this distance is significantly smaller than the expected length of the shortest vector in the lattice. The hidden vector representing the private key becomes the unique closest vector to $\mathbf{c}$, making it recoverable with CVP estimation algorithms like Babai's Nearest Plane.

4 Experimental results and analysis

We conducted a series of experiments on the secp256k1 curve using the synthetic private key $\alpha =$ `0x123456789ABCDEF0123456789ABCDEF0`. Signatures were generated with controlled LSB leakage of ℓ bits per nonce, and the lattice attack was run using varying numbers of signatures d.

Effect of leakage size ℓ

Table 1 summarises the observed outcomes across different parameter settings.

ℓ (bits leaked)	d (signatures)	Lattice dimension	Outcome
8	100	101×101	**Success**
8	24	25×25	Inconsistent (sometimes fails)
3	100	101×101	Failure

Table 1: Attack outcomes for various leakage and sample parameters on secp256k1.

Analysis

The experiments show a clear dependence on both the number of leaked bits ℓ and the number of available signatures d. In all cases, the attack attempts to recover the private key by converting the partially known nonces into approximate modular relations and then solving the resulting closest vector problem using LLL followed by Babai's nearest-plane algorithm.

For $\ell = 8$ and $d = 100$, the attack succeeds reliably in our controlled experiments. In this setting, each signature leaks enough information to make the unknown error term sufficiently small, while the number of signatures gives enough independent relations involving the same private key. As a result, the hidden lattice vector corresponding to the correct private key is close enough to the target vector for Babai's algorithm to recover it after LLL reduction.

The case $\ell = 8$ and $d = 24$ is less stable. Although the leakage per signature is the same, the smaller number of signatures provides fewer constraints on the private key. Consequently, the hidden vector is not always sufficiently distinguished from other lattice vectors, and the quality of the LLL-reduced basis becomes decisive. This explains why the attack may fail even though it succeeds with the same leakage size when more signatures are available.

The failure for $\ell = 3$ is also instructive. In this case, the approximation error is much larger: the error bound is about $q/2^4$, which is very large for a 256-bit group order. Even with $d = 100$ signatures, the corresponding closest vector problem remains too hard for our basic LLL + Babai implementation. Increasing the number of signatures can provide more equations, but it also increases the lattice dimension, which makes lattice reduction more difficult. Thus,

there is a trade-off: more signatures give more information, but they also produce a harder high-dimensional lattice problem.

This illustrates an important gap between theoretical guarantees and practical implementations. Nguyen and Shparlinski prove polynomial-time recovery when roughly $\sqrt{\log q}$ nonce bits are known, up to logarithmic factors and under distributional assumptions. For secp256k1, $\log_2 q \approx 256$, so $\sqrt{\log_2 q} \approx 16$. This should not be interpreted as an exact threshold, but rather as an asymptotic indication of the amount of leakage needed for provable recovery. Our successful experiments with $\ell = 8$ show that, in a controlled synthetic setting, practical recovery can occur below this conservative theoretical regime. On the other hand, the failure at $\ell = 3$ confirms that very small leakage is not automatically exploitable with a basic implementation.

The results also highlight the dependence of the attack on the quality of the lattice reduction step. LLL is efficient and easy to use, but it only provides an approximate reduction of the lattice basis. Babai's nearest-plane algorithm then relies heavily on this reduced basis if the basis is not sufficiently orthogonal, rounding errors accumulate and the algorithm may return the wrong lattice vector. This is exactly what we observe in the hardest setting. Therefore, the success of the attack depends not only on the amount of leaked nonce information, but also on the strength of the lattice reduction algorithm used.

In all successful cases, the recovered candidate $\hat{\alpha}$ satisfies $\hat{\alpha} \equiv \alpha \pmod{q}$ and passes verification against the full set of generated signatures. This confirms that the implementation recovers the actual long-term signing key, not merely a value that fits one particular signature.

5 Implementation challenges

The main challenge was bridging the gap between the theoretical guarantees of Nguyen and Shparlinski and a working implementation. The paper's conditions are asymptotic: roughly $\sqrt{\log q}$ nonce bits are required for the polynomial-time guarantee, up to logarithmic factors, together with a sufficient number of signatures. Determining concrete parameter values for secp256k1 therefore required experimentation.

A second difficulty was the integrality constraint on the LLL basis. The Boneh–Venkatesan lattice naturally contains a rational entry $1/q$ in the last position. We resolved this by scaling the entire basis by q, replacing the rational row with integer entries $(qt_1, \ldots, qt_d, 1)$ and multiplying the diagonal blocks by q^2. This is mathematically equivalent but allows SageMath's integer LLL implementation to be used directly.

Finally, the $\ell = 3$ setting consistently failed despite large numbers of signatures. This reflects a fundamental limitation of basic LLL: when the approximation error is too large relative to q, LLL cannot shorten the basis vectors enough for Babai's algorithm to return the correct coset representative.

6 Discussion

Our experiments reproduce the core mechanism described in the literature. Partial information about ECDSA nonces can be transformed into a Hidden Number Problem instance, and lattice reduction can then recover the long-term private key. This confirms the central message of Nguyen and Shparlinski: the security of ECDSA does not depend only on the hardness of the elliptic-curve discrete logarithm problem, but also on the quality and secrecy of the per-signature nonce.

The successful $\ell = 8, d = 100$ experiment is particularly important because it shows that full

private-key recovery is possible in a concrete implementation, even when only a small number of bits of each nonce is known. This demonstrates the practical danger of seemingly minor leakage. A few leaked bits per signature may appear harmless in isolation, but across many signatures they create enough structure for lattice methods to exploit.

The unstable behavior for $\ell = 8, d = 24$ also provides useful insight. It shows that nonce leakage alone is not the only parameter controlling the attack. The number of available signatures is equally important. With too few signatures, the attacker may not obtain enough independent equations to isolate the private key. This is consistent with the theoretical analysis, where the number of required samples grows with the size of the group and the amount of leakage.

The failed $\ell = 3$ experiment is also meaningful. It shows the limits of our current reproduction rather than contradicting the theory. With only three leaked bits per nonce, the unknown error remains very large. Basic LLL reduction followed by Babai's nearest-plane algorithm is not strong enough to solve the resulting closest vector instance in our setting. More advanced techniques such as BKZ reduction or embedding strategies may improve the situation, but they were outside the scope of our implementation.

Our reproduction is therefore best understood as a controlled demonstration of the attack principle, not as a full-scale reproduction of real-world attacks such as those of Breitner and Heninger. Their work required collecting and processing large numbers of public signatures from blockchains and Internet-wide scans, identifying biased nonce patterns, and then applying lattice attacks to real keys. In contrast, our experiments use synthetically generated signatures where the leakage model is known in advance. This makes the experiment easier to control and verify, but it does not reproduce the full data-collection and filtering pipeline needed for real-world exploitation.

Another limitation is that our implementation focuses on known least-significant-bit leakage. Other leakage models, such as known most-significant bits, shared prefixes, shared suffixes, or short nonces, are also important in practice. Breitner and Heninger observed several such patterns in real cryptocurrency signatures. Extending the implementation to support these leakage models would make the reproduction closer to practical attacks.

The experiments also show that lattice attacks are highly sensitive to parameter choices. The leakage size, number of signatures, lattice dimension, scaling strategy, and reduction algorithm all influence success. This makes the attack more subtle than simply "leaking a few bits breaks ECDSA". A more accurate conclusion is that partial leakage creates exploitable algebraic structure, but successful recovery depends on whether this structure can be made visible to the lattice reduction algorithm.

Two possible improvements follow naturally from our observations:

- **Stronger lattice reduction.** Replacing LLL with stronger reduction algorithms such as BKZ could improve the quality of the reduced basis. A better basis would make Babai's nearest-plane algorithm more likely to recover the correct closest vector, especially in harder instances with smaller nonce leakage.
- **Embedding strategy.** Instead of solving the Closest Vector Problem directly with Babai's algorithm, one can use an embedding technique that transforms the CVP instance into a Shortest Vector Problem instance in a slightly higher-dimensional lattice. This can make the hidden vector corresponding to the secret key appear as one of the short vectors after lattice reduction.

These improvements were not implemented in our reproduction, but they help explain why our

basic implementation fails for very small leakage. They also show that the practical success of the attack depends not only on the amount of leaked information, but also on the strength of the lattice reduction method used.

From a defensive perspective, the main lesson is that nonce generation must be treated as a critical part of ECDSA security. Deterministic nonce generation, such as RFC 6979, reduces the risk of failures caused by weak randomness. However, deterministic nonces do not automatically eliminate all side-channel risks. If the computation of the nonce or the signing operation leaks information, lattice-based attacks may still become relevant. This reinforces the broader conclusion of the project: cryptographic schemes can be mathematically secure under ideal assumptions but still fail when implementation details leak partial information.

7 References

Papers related to our research:

- D. Boneh and R. Venkatesan, *Hardness of Computing the Most Significant Bits of Secret Keys*, CRYPTO 1996.
- N. A. Howgrave-Graham and N. P. Smart, *Lattice Attacks on Digital Signature Schemes*, Designs, Codes and Cryptography, 2001.
- P. Q. Nguyen and I. E. Shparlinski, *The Insecurity of the Digital Signature Algorithm with Partially Known Nonces*, Journal of Cryptology, 2002.
- P. Q. Nguyen and I. E. Shparlinski, *The Insecurity of the Elliptic Curve Digital Signature Algorithm with Partially Known Nonces*, Designs, Codes and Cryptography, 2003.
- J. Breitner and N. Heninger, *Biased Nonce Sense: Lattice Attacks against Weak ECDSA Signatures in Cryptocurrencies*, 2019.